

UNIVERZA V LJUBLJANI  
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Matjaž Kralj  
**Vzorec diplomskega dela**

DIPLOMSKO DELO

ODKOMENTIRAJTE PRAVI PROGRAM IN ODSTRANITE TO  
VRSTICO

MENTOR: viš. pred./doc./izr. prof./prof. dr. Peter Klepec  
SOMENTOR: viš. pred./doc./izr. prof./prof. dr. Martin Krpan

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani [creativecommons.si](http://creativecommons.si) ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

*Besedilo je oblikovano z urejevalnikom besedil  $\text{\LaTeX}$ .*

**Kandidat:** Matjaž Kralj

**Naslov:** Vzorec diplomskega dela

**Vrsta naloge:** npr. Diplomska naloga na visokošolskem programu prve stopnje

**Mentor:** viš. pred./doc./izr. prof./prof. dr. Peter Klepec

**Somentor:** viš. pred./doc./izr. prof./prof. dr. Martin Krpan

**Opis:**

Besedilo teme diplomskega dela študent prepíše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

**Title:** Diploma thesis template

**Description:**

The student shall transcribe the text of the diploma thesis topic from the student information system, where it was entered by the mentor. In a few sentences, they will describe what is expected of the candidate's thesis. What the objectives are, what methods should be used, and perhaps they will also list the key literature.



*Na tem mestu zapišite, komu se zahvaljujete za pomoč pri izdelavi diplomske naloge oziroma pri vašem študiju nasploh. Pazite, da ne boste koga pozabili. Utegnil vam bo zameriti. Temu se da izogniti tako, da celotno zahvalo izpustite.*



Svojemu dragemu računalniku.





# Kazalo

**Povzetek**

**Abstract**

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Uvod</b>                                                                | <b>1</b>  |
| <b>2</b> | <b>Struktura diplomske naloge</b>                                          | <b>3</b>  |
| 2.1      | Slogovne smernice in priporočila . . . . .                                 | 5         |
| <b>3</b> | <b>Osnovni gradniki L<sup>A</sup>T<sub>E</sub>Xa in nasveti za pisanje</b> | <b>7</b>  |
| 3.1      | Koristni nasveti pri pisanju v L <sup>A</sup> T <sub>E</sub> Xu . . . . .  | 8         |
| 3.2      | Pisave v L <sup>A</sup> T <sub>E</sub> Xu . . . . .                        | 9         |
| 3.3      | Skladnost s standardom PDF/A . . . . .                                     | 9         |
| 3.4      | Uporaba količin in enot . . . . .                                          | 10        |
| 3.5      | Sklici in citiranje . . . . .                                              | 11        |
| <b>4</b> | <b>Pogoste napake pri pisanju v slovenščini</b>                            | <b>13</b> |
| <b>5</b> | <b>Plovke: slike in tabele</b>                                             | <b>17</b> |
| 5.1      | Formati slik . . . . .                                                     | 18        |
| 5.2      | Podnapisi k slikam in tabelam . . . . .                                    | 19        |
| 5.3      | Vključevanje več slik in tabel hkrati . . . . .                            | 19        |
| <b>6</b> | <b>Vključevanje programske kode in algoritmov</b>                          | <b>25</b> |
| 6.1      | Vključevanje algoritmov (psevdokode) . . . . .                             | 26        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>7</b> | <b>Matematično okolje in sklicevanje na besedilne konstrukte</b> | <b>29</b> |
| 7.1      | Matrike . . . . .                                                | 30        |
| 7.2      | Razbijanje dolgih enačb . . . . .                                | 31        |
| 7.3      | Sistemi enačb in primeri . . . . .                               | 31        |
| 7.4      | Splošna tipografska pravila v matematiki . . . . .               | 31        |
| <b>8</b> | <b>Kaj pa literatura?</b>                                        | <b>33</b> |
| 8.1      | Zbiranje virov za seznam literature . . . . .                    | 34        |
| <b>9</b> | <b>Sklepne ugotovitve</b>                                        | <b>37</b> |
|          | <b>Literatura</b>                                                | <b>39</b> |

# Povzetek

**Naslov:** Vzorec diplomskega dela

**Avtor:** Matjaž Kralj

V vzorcu je predstavljen postopek priprave diplomskega dela z uporabo okolja L<sup>A</sup>T<sub>E</sub>X. Vaš povzetek mora sicer vsebovati med približno 100 in 200 besed, ta tukaj je odločno prekratek. Dober povzetek je kot mini diploma in vključuje: (1) kratek opis stanja tehnike in kaj so glavne težave oz. opis obravnavanega problema, (2) kako ste ta problem naslovili oz. kratek opis vašega pristopa za reševanje tega problema in (3) (najbolj uspešen) rezultat ali prispevek diplomske naloge.

**Ključne besede:** računalnik, procesor, pomnilnik.



# Abstract

**Title:** Diploma thesis template

**Author:** Matjaž Kralj

This sample document presents the process of preparing a thesis using the L<sup>A</sup>T<sub>E</sub>X environment. Your abstract should otherwise contain between approximately 100 and 200 words; the one provided here is far too short. A good abstract is like a mini-thesis and includes: (1) a brief description of the state of the art and the main issues or a description of the problem addressed, (2) how you addressed this problem or a brief description of your approach to solving it, and (3) the (most successful) result or contribution of the thesis.

**Keywords:** computer, processor, memory.



# Poglavje 1

## Uvod

Prvi koristen nasvet v zvezi uporabo  $\text{\LaTeX}$ a pri pisanju vašega diplomskega dela je, da v celoti preberete ta dokument!

Datoteka na kratko opisuje, kako se pisanja diplomskega dela lotimo z uporabo programskega okolja  $\text{\LaTeX}$  [8, 9].

$\text{\LaTeX}$  omogoča logično urejanje besedil, ki ima v primerjavi z vizualnim urejanjem številne prednosti, saj se problema urejanja besedil loti s programerskega stališča. Logično urejanje besedil omogoča večjo konsistentnost, uniformnost in prenosljivost besedil, predvsem pa nam omogoča lomljenje besed in posledično lepo obojestransko poravnavo. Obojestranska poravnava v vizualnih urejevalnikih povzroči kupe odvečnih in neenakomernih presledkov.

Poglavje 2 na kratko predstavi tipične sestavne dele diplomske naloge in slogovne smernice. V 3. poglavju spoznamo osnovne gradnike  $\text{\LaTeX}$ a, nasvete za pisanje, standard PDF/A, uporabo količin in enot ter pravila za sklice in citiranje. V 4. poglavju opozarjamo na najpogostejše slovnične in tipografske napake, ki jih delamo v slovenščini. Poglavje 5 predstavi vključevanje plovk: vstavljanje in formatiranje posameznih slik in tabel ter postavljanje večih elementov hkrati. V 6. poglavju spoznamo vključevanje programske kode in algoritmov. V 7. poglavju so predstavljeni matematično okolje, zapis matrik in sistemov enačb, besedilni konstrukti (kot so izreki in dokazi) ter sklicevanje nanje. V 8. poglavju se bomo srečali z iskanjem virov in sklicevanjem na

literaturo z uporabo BIB $\text{\LaTeX}$ a. Na koncu sledijo še sklepne ugotovitve z opomnikom za preverjanje pred oddajo naloge.

Ta vzorec ni priročnik za uporabo  $\text{\LaTeX}$ a, saj razloži le nekatere osnovne ukaze, druge funkcionalnosti pa le omeni. Kako se jih uporablja pa naj bralec poišče drugje.



## Poglavje 2

# Struktura diplomske naloge

Diplomska naloga ima sicer predvideno okvirno strukturo, dejanska struktura pa je precej odvisna od področja v katerem delujete, zato se glede tega nujno posvetujte z mentorjem. Ustaljena struktura sicer bralcu, predvsem pa mentorju in članom komisije, omogoča hitro navigacijo in preverjanje znanstvene integritete opravljenega dela.

Tipična struktura diplomske naloge na FRI obsega naslednje obvezne in priporočene gradnike:

**Naslovna stran** poleg naslova v slovenščini in angleščini vsebuje še ime avtorja, mentorja, morebitnega somentorja ter letnico in kraj izdaje. Naslov mora biti jedrnat, a hkrati dovolj informativen, da bralec takoj prepozna domeno računalništva, ki jo naloga obravnava.

**Izjava o avtorstvu** je formalni dokument, s katerim študent jamči, da je diplomsko delo plod lastnega dela in da so vsi uporabljeni viri ustrezno navedeni.

**Povzetek (slovenski in angleški)** je kratek, samostojen tekst (običajno omejen na približno 250 besed), ki povzame motivacijo, uporabljene metode in najpomembnejše rezultate. Na FRI je obvezen tudi prevod povzetka v angleščino (*Abstract*), kar povečuje vidnost dela v mednarodnem okolju.

**Ključne besede** so izbrani termini, ki se uporabljajo za indeksiranje naloge v digitalnih knjižnicah. Izbrati velja besede, ki specifično opisujejo tehnologije, algoritme ali področja (npr. strojno učenje, ogrodje React, generativni modeli).

**Kazalo** vsebine.  $\LaTeX$  vsa kazala generira samodejno, kar zagotavlja, da so številke strani vedno usklajene z dejansko vsebino.

**Uvodno poglavje** pripravi teren za razumevanje problema. V njem jasno opredelimo stanje področja, motivacijo za delo, cilje naloge in hipoteze, ki jih želimo preveriti. Uvod se običajno zaključi s kratko predstavitevijo vsebine po posameznih poglavjih, kar bralcu služi kot zemljevid. Pozorni bodite, da ne pišete števil poglavij ročno, ampak jih referencirate z `\ref{xx}` in poskrbite, da imate pri naslovih poglavjih `\label{xx}`.

**Pregled področja in sorodnih del** je namenjen umeščanju naloge v obstoječ kontekst. Tukaj predstavite katere rešitve za izbrani problem že obstajajo in v čem se vaša rešitev razlikuje od tistih, ki so jih razvili drugi avtorji. Temeljiti pregled literature pokaže vašo razgledanost na področju.

**Osrednja poglavja** tvorijo hrbtenico naloge. Pri računalniških diplomah tipično zajemajo sledeča poglavja, vendar pa jih lahko razbijate na več ločenih poglavij in podpoglavij, jih poimenujete po svoje ali odstranite, povsem odvisno od področja v katerem delujete. O tem se predvsem posvetujte z mentorjem:

- **Metodologija oz. teoretične osnove:** Razlaga konceptov, potrebnih za razumevanje rešitve.
- **Analiza in načrtovanje:** Opis arhitekture rešitve, omejitev, zahtev ipd.
- **Implementacija:** Opis rešitve, podrobnosti razvoja, uporabljene knjižnice in reševanje specifičnih tehničnih izzivov, vendar brez dejanskih števil.

- **Eksperimenti, evalvacija in rezultati:** Testiranje rešitve, primerjava zmogljivosti ali uporabniška študija, predstavitev rezultatov in argumentacija. Tipično se to razbije na več poglavij, odvisno od količine izvedenih eksperimentov.

**Zaključek** povzame dosežene cilje in poda kritično presojo opravljenega dela. Pomembno je, da izpostavite omejitve svoje rešitve in nakažete smeri za morebitno nadaljnje delo ali izboljšave.

**Literatura in viri** je seznam vseh strokovnih in znanstvenih člankov, knjig in spletnih virov, na katere se sklicujete. Vsak vir v seznamu mora imeti ustrezen sklic v besedilu.

**Priloge** so namenjene gradivu, ki bi preveč motilo tok besedila v osrednjem delu, a je pomembno za popolno razumevanje (npr. obsežni kosi programske kode, vprašalniki ali podrobni diagrami).

## 2.1 Slogovne smernice in priporočila

Zavedajte se, da vrstni red branja ne sledi vrstnemu redu pisanja zaključnega dela. Verjetno se boste lotili pisanja sorodnih del in metodologije pred eksperimenti ali rezultati, vsekakor pa pred uvodnim razdelkom ali povzetkom, ki ga zagotovo napišite povsem na koncu. Tega se zavedajo tudi bralci, zato se vam pri pisanju uvoda ni potrebno pretvarjati, da eksperimenti še niso bili opravljeni.

- Strokovna besedila morajo biti nedvoumna, zato naj bodo stavki kratki, preprosti in razumljivi, terminologija dosledna.
- Pri pisanju zaključnega dela upoštevajte akademski slog, ki se močno razlikuje od poljudnega pisanja. Besedilo pišite v prvi osebi množine (npr. „razvili smo“, „opazimo“), saj s tem poudarite objektivnost raziskovalnega procesa, v katerega sta vključena študent in mentor. Dovoljena je tudi uporaba trpnika, ki pa je jezikovno okornejša („razvit je bil“,

„izdelano je bilo“). Prve osebe ednine („razvil sem“, „izdelal sem“) ne uporabljajte oziroma jo uporabite izjemoma samo tam, kjer želite poudariti svojo osebno odločitev med več možnostmi.

- Vse trditve v nalogi morajo biti preverljive. To pomeni, da morajo temeljiti na lastnih izpeljavah ali testiranjih, če pa povzimate ugotovitve drugih, je navajanje vira obvezno.
- Za posamezen pojem ali objekt uporabljajte v celotnem delu enako ime in oznako (če je potrebna).
- Če vpeljujete nove pojme, pazite, da so nedvoumno in natančno definirani.
- Uporabljajte uveljavljeno strokovno terminologijo.
- S pisanjem strokovnih besedil se razvija tudi slovenska strokovna terminologija. Pri strokovnih izrazih, ki še nimajo prevoda v slovenščino, po potrebi in v sodelovanju z mentorjem poiščite ustrezen slovenski prevod.
- Posebna pozornost naj velja prevodom iz angleščine. Pazite na slovenska slovnična pravila. Samostalniški prilastki so v slovenščini na desni strani osebka ali predmeta (SQL query → poizvedba SQL, API interface → vmesnik API,  $x$ -axis → os  $x$ ), obratni vrstni red, kot je na primer SQL poizvedba, Windows strežnik ali ACME sistem, je napačen.

## Poglavje 3

# Osnovni gradniki L<sup>A</sup>T<sub>E</sub>Xa in nasveti za pisanje

L<sup>A</sup>T<sub>E</sub>X bi lahko najbolj preprosto opisali kot programski jezik namenjen oblikovanju besedil. Tako kot vsak visokonivojski programski jezik ima tudi L<sup>A</sup>T<sub>E</sub>X številne ukaze za oblikovanje besedila in okolja, ki omogočajo strukturiranje besedila.

Vsi L<sup>A</sup>T<sub>E</sub>Xovi ukazi se začnejo z levo poševnico `\`, okolja pa definiramo bodisi s parom zavitih oklepajev `{ in }` ali z ukazoma `\begin{ }` in `\end{ }`. Ukazi imajo lahko tudi argumente, obvezni argumenti so podani v zavutih oklepajih, opcijski argumenti pa v oglatih oklepajih.

Z ukazi torej definiramo naslov in imena avtorjev besedila, poglavja in podpoglavja in po potrebi bolj podrobno strukturiramo besedila na spiske, navedke itd. Posebna okolja so namenjena zapisu matematičnih izrazov, kratki primeri so v naslednjem poglavju.

Vse besedilne konstrukte lahko poimenujemo in se s pomočjo teh imen nato kjerkoli v besedilu na njih tudi sklicujemo.

L<sup>A</sup>T<sub>E</sub>X sam razporeja besede v odstavke tako, da optimizira razmike med besedami v celotnem odstavku. Nov odstavek začnemo tako, da izpustimo v izvirnem besedilu prazno vrstico. Da besedilo skoči v novo vrstico pa ukažemo z dvema levima poševnicama. Število presledkov med besedami v

izvornem besedilo ni pomembno.

### 3.1 Koristni nasveti pri pisanju v L<sup>A</sup>T<sub>E</sub>Xu

Programski paket L<sup>A</sup>T<sub>E</sub>X je bil prvotno predstavljen v priročniku [8] in je v resnici nadgradnja sistema T<sub>E</sub>X avtorja Donalda Knutha [18], znanega po svojih knjigah o umetnosti programiranja ter Knuth-Bendixovem algoritmu [7]. T<sub>E</sub>X in njegove izpeljanke so odprtokodni programi.

Različnih implementacij L<sup>A</sup>T<sub>E</sub>Xa je cela vrsta, a priporočamo spletno verzijo, ki poenostavi sodelovanje pri pisanju, Overleaf.

Včasih smo si pri pisanju v L<sup>A</sup>T<sub>E</sub>Xu pomagali predvsem s tiskanimi priročniki [8], danes pa je enostavneje in hitreje, da ob vsakem problemu za pomoč enostavno povprašamo najljubši jezikovni model.

L<sup>A</sup>T<sub>E</sub>X včasih ne zna pravilno deliti slovenskih besed, ki vsebujejo črke s strešicami. Če taka beseda štrli preko desnega roba, lahko L<sup>A</sup>T<sub>E</sub>Xu pokažemo, kje se tako besedo deli takole: `ra\~{c}u\~{n}al\~{n}i\~{s}tvo`. Katere vrstice so predolge lahko vidimo tako, da dokument prevedemo s vključeno opcijo `draft`: `\documentclass[a4paper, 12pt, draft]{book}`.

V izvornem besedilu lahko začnate vsak stavek v novi vrstici, saj L<sup>A</sup>T<sub>E</sub>X sam razporeja besede po vrsticah postavljenega besedila. Na ta način je tudi iskanje po izvornem besedilu in popravljanje veliko hitrejše. Večina sistemov za T<sub>E</sub>Xiranje sicer omogoča s klikanjem enostavno prestopanje iz prevedenega besedila na ustrezno mesto v izvornem besedilu in obratno.

Boljšo preglednost dosežemo tako kot pri pisanju programske kode – z vizualnim urejanjem kode in izpuščanjem praznih vrstic. Pri spreminjanju in dodajanju izvirnega besedila je najbolje pogosto prevajati, da se sproti prepričamo, če so naši nameni pravilno izpolnjeni.

Kadar besedilo, ki je že bilo napisano z nekim vizualnim urejevalnikom (npr. z Wordom), želimo prenesti v L<sup>A</sup>T<sub>E</sub>X, je tudi najbolje to delati postopoma s posameznimi bloki besedila, tako da lahko morebitne napake hitro identificiramo in odpravimo. Za prevajanje Wordovih datotek v L<sup>A</sup>T<sub>E</sub>X – in

obratno – sicer obstajajo prevajalniki, ki pa običajno ne generirajo tako čisto logične strukture besedila, kot jo sicer  $\text{\LaTeX}$  omogoča. Hiter in enostaven način prevedbe besedila, ki zahteva sicer ročne dopolnitve, lahko poteka tudi tako, da besedilo urejeno z vizualnim urejevalnikom najprej shranimo v formatu pdf, nato pa to besedilo uvozimo v urejevalnik, kjer urejamo izvirno besedilo v formatu  $\text{\LaTeX}$ .

## 3.2 Pisave v $\text{\LaTeXu}$

V  $\text{\LaTeX}$ ovem okolju lahko načeloma uporabljamo poljubne pisave. Izbira poljubne pisave pa ni tako enostavna kot v vizualnih urejevalnikih besedil. Posamezne oblikovno medseboj usklajene pisave so običajno združene v družine pisav. V  $\text{\LaTeXu}$  se privzeta družina pisav imenuje Computer Modern, kjer so poleg navadnih črk (roman v  $\text{\LaTeXu}$ ) na voljo tudi kurzivne črke (*italic* v  $\text{\LaTeXu}$ ), krepke (**bold** v  $\text{\LaTeXu}$ ), kapitelke (SMALL CAPS v  $\text{\LaTeXu}$ ), linearne črke (**san serif** v  $\text{\LaTeXu}$ ), pisava pisalnega stroja (**typewriter** v  $\text{\LaTeXu}$ ) in nekatere njihove kombinacije, npr. krepke linearne črke (**san serif** v  $\text{\LaTeXu}$ ). V istem dokumentu zaradi skladnega izgleda uporabljamo običajno le pisave ene družine. Pomembna je tudi konsistentna raba večih pisav in da ne pretiravamo z mešanjem različnih pisav.

Ko začnemo uporabljati  $\text{\LaTeX}$ , je zato najbolj smiselno uporabljati kar privzete pisave, s katerimi je napisan tudi ta dokument. Z ustreznimi ukazi lahko nato preklapljamo med navadnimi, kurzivnimi, krepkimi in drugimi pisavami. Zelo enostavna je tudi izbira velikosti črk.  $\text{\LaTeX}$  odlično podpira večjezičnost, tudi v sklopu istega dokumenta, saj obstajajo pisave za praktično vse jezike, tudi take, ki ne uporabljajo latinskih črk.

## 3.3 Skladnost s standardom PDF/A

Repozitorij Univerze v Ljubljani zahteva, da so oddana zaključna dela skladna s standardom PDF/A. V predlogi smo zagotovili skladnost s standardom

PDF/A-2b. S predlogo žal ne moremo preprečiti, da bi uporabnik v diplomsko delo dodal elemente, ki niso skladni s standardom. Pred oddajo zaključnega dela zato uporabnikom priporočamo preverjanje skladnosti z ustreznimi preverjevalniki, npr. z orodjem veraPDF<sup>1</sup>.

### 3.4 Uporaba količin in enot

Pri tehničnem in strokovnem pisanju je zelo pomembno dosledno razlikovanje med količinami in njihovimi pripadajočimi merskimi enotami. Neustrezen zapis lahko vodi do napačne interpretacije podatkov in zmanjša strokovno vrednost dokumenta.

**Količina** predstavlja lastnost sistema, strojne ali programske opreme, ki jo je mogoče kvantitativno določiti. Simbole količin (npr.  $C$  za kapaciteto pomnilnika,  $f$  za frekvenco procesorja,  $B$  za pasovno širino) pišemo praviloma v **ležeči pisavi**.

**Merska enota** je dogovorjena vrednost, s katero primerjamo isto vrsto količine. Simbole enot (npr. B, Hz,  $\text{bit s}^{-1}$ ) pišemo v **pokončni pisavi**.

Med številčno vrednostjo in pripadajočo enoto mora biti vedno prisoten **stični presledok** (tilda), tako kot pri citatih. To zagotavlja, da vrednost in enota ob morebitnem prelomu vrstice ostaneta skupaj.

Za dosego standardiziranega zapisa v okolju L<sup>A</sup>T<sub>E</sub>X uporabljamo paket `siunitx`. Ta omogoča avtomatsko upravljanje s presledki in decimalnimi ločili. Spodaj so navedeni primeri pravilne uporabe:

- **Izpis vrednosti z enoto:** Ukaz `\qty{100}{\giga\byte}` vrne 100 GB.
- **Posebni znaki:** Odstotke in stopinje pišemo s presledkom (za to že poskrbi L<sup>A</sup>T<sub>E</sub>X), npr.:

– `\qty{98}{\percent}` vrne 98 %,

---

<sup>1</sup><https://verapdf.org/>



– `\qty{85}{\degreeCelsius}` vrne  $85\text{ }^{\circ}\text{C}$ .

V slovenščini kot decimalno ločilo uporabljamo vejico, kar v paketu `siunitx` nastavimo globalno z ukazom `output-decimal-marker = {,}`.

Še primer rabe enot v besedilu: Pri obremenitvenem testu strežnika smo ugotovili, da pri frekvenci  $f = 3,5\text{ GHz}$  in obremenitvi  $98\%$  prepustnost omrežja doseže  $B = 1,2\text{ Gbit s}^{-1}$ . Vse meritve so bile opravljene pri temperaturi procesorja  $T = 85\text{ }^{\circ}\text{C} \pm 1,5\text{ }^{\circ}\text{C}$ .

### 3.4.1 Računalnikarji, spoštujte SI!

Pri pisanju se zavedajte, da cel svet, z izjemo Liberije, Mjanmarja in Združenih držav Amerike, uradno uporablja Mednarodni sistem enot (SI), ki nam vsakodnevno lajša življenje. Ali ni čudovito reči, da ste letos prevozili  $23\text{ Mm}$ , namesto nepoučenega dolgovezenja s  $23\,000\text{ km}$ ?

Pri zapisovanju velikosti v bajtih prihaja zaradi zgodovinskih razlogov do pogostih težav. Velja sledeče:

- Če ste uporabili desetiški pretvornik (1000), potem uporabite SI predpone: k (kilo), M (mega), G (giga), T (tera), tj. kB, MB, GB, TB.
- Če ste uporabili dvojiški pretvornik (1024), potem uporabite dvojiške predpone: kibi (Ki), mebi (Mi), gibi (Gi), tebi (Ti), tj. KiB, MiB, GiB, TiB. Naj vas ne zmoti dejstvo, da se veliko proizvajalcev in celo določeni operacijski sistemi tega ne držijo (lahko da prihajajo iz katere od treh posebnih držav).

Seveda ne pozabite tudi na razliko v zapisu bajta B in bita bit!

## 3.5 Sklici in citiranje

Pri pisanju strokovnih besedil je nujno dosledno sklicevanje na slike, tabele, enačbe in druga poglavja ter citiranje uporabljene literature. Sistem  $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$

nam to močno olajša, saj ob prevajanju samodejno poskrbi za pravilno oštevilčenje. Uporabljamo naslednje osnovne ukaze:

- **Označevanje** (`\label`): Element, na katerega se želimo sklicevati, moramo najprej označiti z ukazom `\label{ime_oznake}`. Priporočljivo je, da za boljšo preglednost uporabljate predpone, na primer `sl:` za slike, `tab:` za tabele, `en:` za enačbe in `pogl:` za poglavja (npr. `\label{sl:galaksija}` ali `\label{ch:osnove}`).
- **Sklicevanje** (`\ref`): V besedilu se na označeni element skličemo z ukazom `~\ref{ime_oznake}`.  $\text{\LaTeX}$  bo na tem mestu izpisal ustrezno številko. Če se sklicujemo na matematično enačbo, namesto tega uporabimo ukaz `~\eqref{ime_oznake}`, ki številko enačbe samodejno obda z okroglimi oklepaji.
- **Citiranje** (`\cite`): Za navajanje virov iz bibliografske datoteke uporabimo ukaz `~\cite{kljuc_vira}`.

**Zelo pomembno pravilo:** Med besedo in samim sklicem ali citatom mora vedno stati **nedeljivi presledek**, ki ga v  $\text{\LaTeX}$ u označimo s tildo (`~`). S tem preprečimo, da bi številka sklica ob prelomu vrstice padla sama v novo vrstico, kar je tipografsko nepravilno.

Pravilna raba v besedilu torej izgleda takole: Kot lahko vidimo na sliki `~\ref{sl:galaksija}`, velja enačba `~\eqref{en:masa}`. Podobno velja za literaturo: Algoritem je opisan v poglavju `~\cite{knuth}`.

## Poglavje 4

# Pogoste napake pri pisanju v slovenščini

Pri pisanju strokovnih besedil v slovenščini se pogosto srečujemo s tipičnimi napakami, ki izvirajo iz vpliva tujih jezikov ali nepoznavanja tipografskih pravil L<sup>A</sup>T<sub>E</sub>Xu. Spodaj so zbrana ključna pravila, organizirana po kategorijah:

- **Vrstni red besed in kratic:** Pridevniki, ki se ne sklanjajo (npr. kratice), morajo v slovenščini stati za samostalnikom. Pravilno pišemo „model CAD“ in **ne** „CAD model“.
- **Sklanjanje tujih imen:** Pri sklanjanju tujih lastnih imen v slovenščini ne uporabljamo vezajev. Pravilno je „Applov operacijski sistem“ in **ne** „Apple-ov“.
- **Ločila in stičnost:** Pika, klicaj in vprašaj so levostični (pred njimi ni presledka, za njimi pa je). Klicajev in vprašajev se v strokovnih besedilih načeloma izogibamo. Oklepaji so desnostični, zaklepaji pa levostični: „(takole)“.
- **Tropičje in naštevanje:** Tropičja enostavno v diplomah ne uporabljajte. V znanstvenih besedilih in diplomah uporabljamo itd. ali idr. in ne tropičja. Pozorni bodite, da lahko uporabite le npr. ali itd. in ne obeh v eni povedi.

Za kasnejšo rabo tropičja v življenju pa vendarle velja sledeče. V slovenščini je tropičje praviloma nestično ločilo. V  $\text{\LaTeX}$ u ga zapišemo z ukazom `\dots` in ne s tremi pikami. Primer: „pes, maček, rosomah, ...“ Izjema je tropičje, ki mu sledi drugo ločilo; takrat je levonestično, npr. „pes, maček, rosomah, ...!“

- **Narekovaji:** So stična ločila. V  $\text{\LaTeX}$ u ne smemo uporabiti zgolj tipke " ali dveh ", saj se začetni in končni narekovaji ne bodo pravilno izrisali. Najboljša praksa je uporaba ukaza `\enquote{...}`, ki samodejno postavi pravilne slovenske narekovaje: „spodaj devetice, zgoraj šestice“.
- **Vezej in pomišljaj:**
  - **Vezej (-):** Je levo in desno stičen, npr. *slovensko-angleški* in označuje enakovredno razmerje med pojmomoma. V kodi ga pišemo z enim znakom -.
  - **Pomišljaj (–):** V slovenščini in v (britanski) angleščini je nestičen (s presledki). V kodi ga zapišemo z dvema znakoma –, npr. *Pozor -- hud pes!*
  - **Razponski pomišljaj (–):** Uporabljamo ga za razpone (npr. „od–do“), je stičen: *strani 7--11* in se ga v kodi piše povsem enako kot pomišljaj (--).
  - **Ameriški pomišljaj (—):** t.i. *em-dash* je daljši, stičen in se v kodi piše s tremi znaki ---, opravlja pa povsem enako vlogo kot nestični pomišljaj v slovenščini oz. v britanski angleščini t.i. *en-dash*, npr. *Beware---vicious dog!* V slovenščini ga seveda ne uporabljajte.
- **Presledki ob ločilih in kraticah:**  $\text{\LaTeX}$  privzeto uporablja t.i. francoski razmik, kar pomeni, da je presledek po koncu stavka malenkost večji od tistega med besedami. To vodi do dveh specifičnih težav:
  - **Kratice sredi stavka:** Če piki za kratiko (npr. *prof.*, *dr.*, *itd.*) sledi presledek, bo  $\text{\LaTeX}$  to interpretiral kot konec stavka in vstavil

predolg razmik. To preprečimo z uporabo tilde ~ (stični presledek) ali poševnice s presledkom \ . Pravilno: `prof.~dr.~Novak` ali `npr.\` v tem primeru.

- **Kratice na koncu stavka:** Če se stavek konča z veliko črko (npr. `kratica na koncu stavka`), `LATEX` misli, da gre za kratico sredi stavka in vstavi premajhen razmik. Za pravilno interpretacijo konca stavka pred piko vstavimo ukaz `\@`.

Pravilno: `Razvili so novo CPE\@`. Sledila je testna faza.

- **Vejice in stavčna zgradba:** Vejica loči stavke (povedke), razen če so zvezani z vezniki *in*, *pa*, *ter*, *ne–ne*, *nití–nití*, *ali*, *bodisi*, *oziroma*. Čeprav velja pravilo „pred ki, ko, ker, da, če, vejica skače“, je potrebno paziti na vrvke in večbesedne veznike. V strokovnih besedilih je bolje pisati krajše in jasne stavke. Veliki jezikovni modeli vam bodo pri postavljanju vejic v veliko pomoč, za zares pravilno postavljanje pa svetujemo uporabo spletnega orodja CJVT vejice (<https://orodja.cjvt.si/vejice/>).
- **Terminologija in viri:** Za nove računalniške pojme si pomagamo z uveljavljenimi slovarji. Glavni viri so *Islovar* Slovenskega društva Informatika [6], Slovar Slovenskega društva za razpoznavanje vzorcev [17] ter osrednje jezikovno spletišče *Fran* [11].



## Poglavje 5

### Plovke: slike in tabele

Slike in daljše tabele praviloma vključujemo v dokument kot plovke. Pozicija plovke v končnem izdelku ni pogojena s tekom besedila, temveč z izgledom strani. L<sup>A</sup>T<sub>E</sub>X bo skušal plovko postaviti samostojno, praviloma na mestu, kjer se pojavi v izvornem besedilu, sicer pa na vrhu strani, na kateri se na takšno plovko prvič sklicujemo. Pri tem pa bo na vsako stran končnega izdelka želel postaviti tudi sorazmerno velik del besedila. V skrajnem primeru, če imamo res preveč plovk na enem mestu besedila, ali če je plovka previsoka, se bo L<sup>A</sup>T<sub>E</sub>X odločil za stran popolnoma zapolnjeno s plovkami.

Poleg tega, da na položaj plovke vplivamo s tem, kam jo umestimo v izvorno besedilo, lahko na položaj plovke na posamezni strani prevedenega besedila dodatno vplivamo z opcijami **here**, **top** in **bottom**. Zelo velike slike je najbolje postaviti na posebno stran z opcijo **page**. Skaliranje slik po njihovi širini lahko prilagodimo širini strani tako, da kot enoto za širino uporabimo kar širino strani, npr. `0.5\textwidth` bo raztegnilo sliko na polovico širine strani. Sliko lahko po potrebi tudi zavrtimo za 90 stopinj in jo razstegnemo na višino strani. Tako bodo podrobnosti na sliki lažje berljive in prostor na strani bo bolje izkoriščen.

Na vse plovke se moramo v besedilu sklicevati, saj kot beseda plovka pove, plovke plovejo po besedilu in se ne pojavijo točno tam, kjer nastopajo v izvornem besedilu. Vendar naj bosta sklic na plovko v besedilu in sama plovka

v oblikovanem besedilu čim bližje skupaj, tako da bralcu ne bo potrebno listati po diplomih. Upoštevajte pa, da se naloge tiska dvostransko in da se hkrati vidi dve strani v dokumentu! Na to, kje se bo slika ali druga plovka pojavila v postavljenem besedilu lahko torej najbolj vplivamo tako, da v izvorni kodi plovko premikamo po besedilu nazaj ali naprej!

Tabele je najboljše oblikovati kar neposredno v  $\text{\LaTeX}$ u, saj za oblikovanje tabel obstaja zelo fleksibilno okolje `tabular`. Slike pa je po drugi strani pogosto najlažje oblikovati oziroma izdelati z drugimi orodji in programi, rezultate shraniti v formatu `.pdf` ali `.jpeg` in nato v  $\text{\LaTeX}$ u le vključiti ustrezno slikovno datoteko. Za pisanje besed, ki so vključene v slike, uporabite pisave/fonte, ki so čimbolj podobne pisavam v samem besedilu, ali še bolje shranite brez fonta in pustite  $\text{\LaTeX}$ u, da vključi svojega.

Knjižnica <https://en.wikibooks.org/wiki/LaTeX/PGF/TikZ> pa omogoča risanje raznovrstnih grafov neposredno v okolju  $\text{\LaTeX}$ .

Na vse tabele in slike se moramo v besedilu sklicevati, saj kot plovke v oblikovanem besedilu niso nujno na istem mestu kot v izvornem besedilu. Pri sklicevanju na slike uporabimo veliko začetnico, npr. "glej Sliko 5.1", saj gre za ime slike.

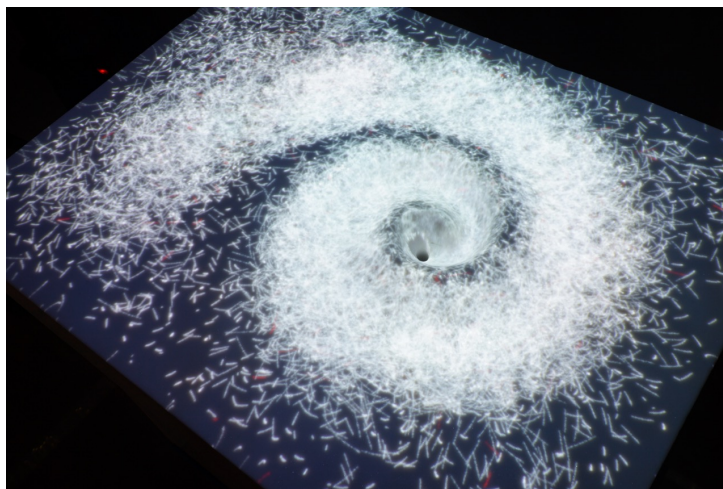
## 5.1 Formati slik

V dokument  $\text{\LaTeX}$  lahko vključimo slike različnih formatov, tako rasterske slike kot vektorske slike.

**Za grafe in diagrame so najbolj primerne vektorske slike** v formatu `.pdf`. Ne samo, da bo besedilo predstavljeno kot besedilo, ampak bodo slike lepe in berljive tudi ob veliki povečavi. Še več, shranjevanje v vektorski obliki od vas običajno ne zahteva nič dodatnega truda. Če npr. graf narišete v Pythonu, le poskrbite, da ga shranite kot `.pdf` in ne kot npr. `.png`, nikakor pa ne kot `.jpg`.

Če vključujete fotografije oz. nekaj česar niste narisali pa bo to verjetno shranjeno v rasterski obliki in shranjeno kot `.jpg` ali `.png`. Slika 5.1 je npr.





Slika 5.1: Virtualno obogatena skulptura [16]. Rezultate računalniško generirane animacije z video projektorjem projeciramo na kamnito skulpturo, da ustvarimo vtis, kot da bi po skulpturi polzele vodne kapljice [14, 15].

v formatu `.jpg`, vendar na povsem enak način vključite tudi npr. `pdf`.

## 5.2 Podnapisi k slikam in tabelam

Vsaki sliki ali tabeli moramo dodati podnapis, ki na pojasnjuje, kaj je na sliki ali tabeli, obrazloži vse morebitne okrajšave, pojasni glavna dognanja. Ne bojte se daljših opisov. Če nekdo le prelista diplomsko delo, naj bi že iz slik in njihovih podnapisov lahko na grobo razbral, kakšno temo naloga obravnava.

Če slike povzamemo iz drugih virov, se moramo v podnapisu k taki sliki nujno sklicevati na ta vir!

## 5.3 Vključevanje več slik in tabel hkrati

Pri pisanju diplomske naloge se pogosto pojavi potreba po prikazovanju več slik ali tabel ene ob drugi. Za ta namen uporabljamo paket `subcaption`, ki predstavlja sodoben in podprt standard v  $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ u.

V spletnih virih in starih predlogah študenti prepogosto najdejo in uporabijo paketa `subfigure` ali `subfig`. Oba paketa sta zastarela in pogosto povzročata težave pri prevajanju ali konflikte z drugimi paketi (npr. pri generiranju hiperpovezav). Teh dveh paketov zato **ne uporabljajte**. Prav tako se za razporejanje slik v mrežo ne poslužujte paketa `TikZ`. Ta paket je sicer izjemno močno orodje, a je namenjen risanju vektorskih shem in je za preprosto razporejanje slik odločno preveč kompleksen.

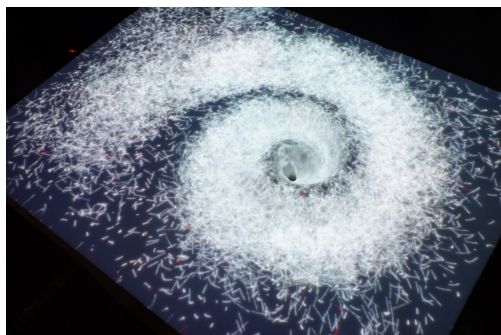
Za uporabo sodobnega pristopa v preambulo dokumenta dodajte ukaz `\usepackage{subcaption}`. Paket nam ponuja dve okolji: `subfigure` za slike in `subtable` za tabele. Obe okolji v ozadju delujeta na podobnem principu kot standardno okolje `minipage` (ki ga v  $\text{\LaTeX}$ u pogosto uporabljamo za postavljanje povsem neodvisnih blokov vsebin enega ob drugega), saj obema podamo eksplicitno širino, ki jo bosta zavzela.

Ker so slike in tabele plavajoči objekti, ne smemo nikoli predpostavljati, kje natančno na strani se bodo izrisale. Na vse elemente se v besedilu vedno sklicujemo izključno z ukazom `\ref`, pred katerim uporabimo nedeljivi presledek (`~`).

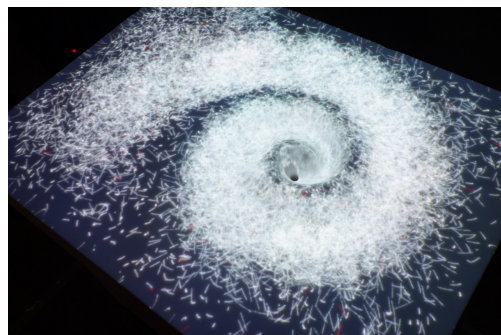
### 5.3.1 Več slik s podnaslovi

Kadar želimo vsaki sliki dodeliti svojo oznako (npr. a, b) in podnaslov, uporabimo okolje `subfigure`. Širino posamezne podslike določimo z ulomkom širine besedila, npr. `0.48\textwidth`. Vmesni prostor do polne širine zapolnimo z ukazom `\hfill`. Na sliki 5.2 lahko vidimo primer dveh slik, ki sta postavljeni ena ob drugi.

Za prehod v novo vrsto znotraj istega plavajočega objekta preprosto izpustimo prazno vrstico in po želji dodamo nekaj vertikalnega prostora z ukazom `\vspace`. Na tak način je zgrajena mreža štirih slik na sliki 5.3.

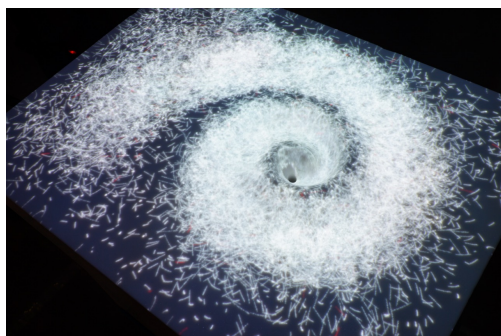


(a) Podnaslov leve slike.

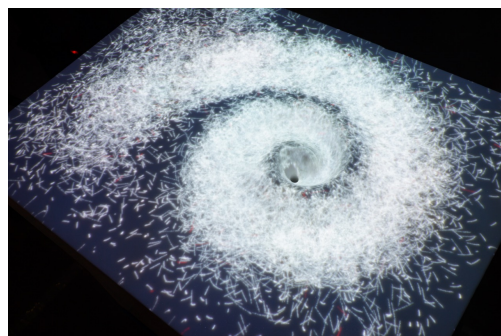


(b) Podnaslov desne slike.

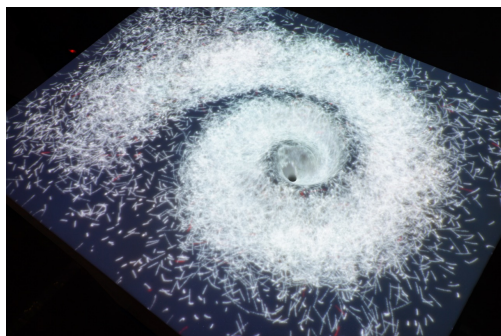
Slika 5.2: Skupni naslov, ki opisuje obe sliki.



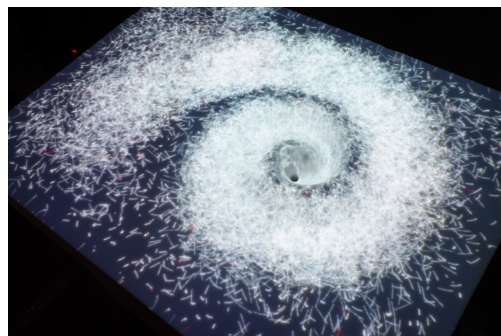
(a) Zgoraj levo.



(b) Zgoraj desno.



(c) Spodaj levo.



(d) Spodaj desno.

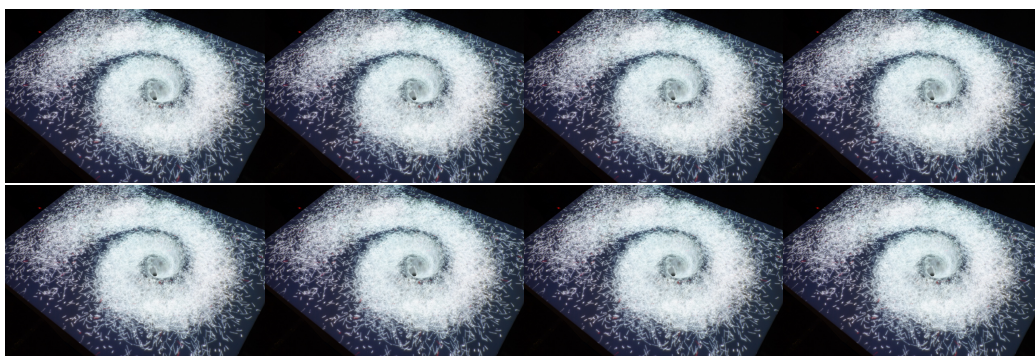
Slika 5.3: Mreža štirih slik s posameznimi podnaslovi.

### 5.3.2 Več slik brez podnaslovov (mreža brez presledkov)

Če individualni podnaslovi niso potrebni in želimo le en skupen naslov celotne mreže slik, okolja `subfigure` sploh ne potrebujemo. Slike preprosto naložimo znotraj navadnega `figure` okolja in določimo njihovo velikost.

Če želimo ustvariti neprekinjeno mrežo povsem **brez vmesnih presledkov** (kot to prikazuje slika 5.4), moramo poskrbeti za dve stvari:

1. **Horizontalni presledki:** Širino določimo tako, da je vsota vseh širin v vrstici točno enaka `1.0\textwidth` (npr. štiri slike s širino `0.25\textwidth`). Na konec vsake vrstice z ukazom `\includegraphics` moramo dodati znak za komentar (%), s čimer preprečimo, da bi  $\text{\LaTeX}$  prelom vrstice interpretiral kot presledek.
2. **Vertikalni presledki:** Znotraj posebnega bloka moramo uporabiti ukaz `\offinterlineskip`, ki v celoti izklopi samodejno vrivanje vertikalnega prostora med vrsticami. Pomembno je, da blok po zadnji sliki zapremo z `}`, preden definiramo `\caption`.



Slika 5.4: Mreža osmih slik brez vmesnih presledkov in z le enim skupnim naslovom.

### 5.3.3 Več tabel ena ob drugi

Logika postavljanja dveh tabel ene ob drugo je popolnoma enaka kot pri slikah, le da znotraj okolja `table` uporabimo okolje `subtable`. Edino pomembno

pravilo, na katerega ne smete pozabiti, je: **podnaslovi tabel vedno sodijo nad tabelo**, medtem ko podnaslovi slik sodijo pod njo.

Če ugotovite, da je vsebina tabele preširoka in povzroča prekrivanje, lahko celotno okolje `tabular` ovijete v ukaz `\resizebox{\linewidth}{!}{...}`, ki tabelo proporcionalno pomanjša, da se natančno prilega širini podtabele. Tabela 5.1 prikazuje ustrezen zapis na primeru vrednotenja modelov računalniškega vida.

Tabela 5.1: Primerjava uspešnosti klasifikacije slik z različnimi arhitekturami globokih nevronske mrež na dveh standardnih podatkovnih zbirkah. Pri metriki *točnost* (angl. *accuracy*) so boljše višje vrednosti (označeno s puščico  $\uparrow$ ), pri metriki *napaka* (angl. *error rate*) pa so boljše nižje vrednosti (označeno s puščico  $\downarrow$ ).

(a) Rezultati na zbirki CIFAR-10.

| arhitektura | točnost ( $\uparrow$ ) | napaka ( $\downarrow$ ) |
|-------------|------------------------|-------------------------|
| ResNet-18   | 93,5 %                 | 6,5 %                   |
| ResNet-50   | 95,2 %                 | 4,8 %                   |
| ViT-Base    | 97,8 %                 | 2,2 %                   |

(b) Rezultati na zbirki ImageNet.

| arhitektura | točnost ( $\uparrow$ ) | napaka ( $\downarrow$ ) |
|-------------|------------------------|-------------------------|
| ResNet-18   | 69,7 %                 | 30,3 %                  |
| ResNet-50   | 76,1 %                 | 23,9 %                  |
| ViT-Base    | 84,2 %                 | 15,8 %                  |



## Poglavje 6

# Vključevanje programske kode in algoritmov

Svojo programsko kodo v poljubnem programskem jeziku najlažje vključimo s paketom `minted`. Okoli bloka `\minted{ }` sicer lahko podamo klasični `\listing{ }` in skupaj z dodanim `\caption` dobimo vse potrebno za prikaz kode, vendar pa se takšen blok ne razteza pravilno skozi več strani. Ena od možnosti za rešitev je uporaba bloka `\center{ }`, za napis pa `\captionof{ }`, kot je prikazano pri obeh primerih izsekov kode 1 in 2.

Izsek kode 1: Kratak primer kode v programskem jeziku Python.

```
1 def python_veverica():  
2     # Oh, kako je to lepo!  
3     x = -1  
4     print("Res izgleda čudovito.")
```

Kar je nadvse čudovito pri paketu `minted` je to, da lahko podamo dele kode znotraj vrstice, kot je npr. tole: `python_veverica()`.

Izsek kode 2: Malo daljši primer programske kode v jeziku Javascript, ki se razteza skozi dve strani.

```
1 async function pridobiUporabnika(id) {
```

```
2  const url = `https://api.primer.fri/users/${id}`;  
3  
4  try {  
5      const odgovor = await fetch(url);  
6  
7      // Preverimo, če je statusni kod v območju 200-299  
8      if (!odgovor.ok) {  
9          console.error(`Napaka pri povezavi: ${odgovor.status}`);  
10         throw new Error(`Uporabnika z ID ${id} ni bilo mogoče  
            ↪ najti.`);  
11     }  
12  
13     const podatki = await odgovor.json();  
14  
15     // Logiranje uspešnega prejema podatkov  
16     console.log("Podatki so bili uspešno prejeti.");  
17     console.log(`Ime uporabnika: ${podatki.name}`);  
18  
19     return podatki;  
20  
21 } catch (napaka) {  
22     // Tukaj ulovimo tako mrežne napake kot napake pri branju JSON-a  
23     console.warn("Prišlo je do nepričakovane težave.");  
24     return { error: napaka.message };  
25 }  
26 }
```

Vodoravni črti pri začetku in koncu izseka, v kombinaciji s sivim ozadjem, vizualno sporočata morebitno raztezanje skozi več strani. To je razvidno pri izseku kode 2.

## 6.1 Vključevanje algoritmov (psevdokode)

Medtem ko paket `minted` uporabljamo za prikaz dejanske implementacije v specifičnem programskem jeziku, v diplomskih nalogah pogosto opisujemo samo logiko reševanja problema na višjem nivoju. Za zapis algoritmov v obliki psevdokode je standardna izbira kombinacija paketov `algorithm` in `algpseudocode` (slednji je del paketa `algorithmicx`).

Za razliko od izsekov izvirne kode, algoritme običajno obravnavamo kot



standardne plavajoče objekte (tako kot slike in tabele). To pomeni, da  $\text{\LaTeX}$  sam poišče optimalno mesto za njihov prikaz na strani. V besedilu se nanje sklicujemo z nedeljivim presledkom in ukazom `\ref`, na primer: Algoritem 1 prikazuje klasičen postopek za iskanje največjega skupnega delitelja.

Naslov (`\caption`) pri algoritmih običajno postavljamo na vrh, tik pod začetek okolja `algorithm`. Številka [1] ob ukazu `\begin{algorithmic}` pa  $\text{\LaTeX}$ u sporoči, naj oštevilči vsako vrstico algoritma, kar nam olajša sklicevanje na posamezne korake v besedilu.

---

**Algorithm 1** Evklidov algoritem za iskanje največjega skupnega delitelja

---

**Require:**  $a \geq 0 \vee b \geq 0$

**Ensure:** Največji skupni delitelj  $a$  in  $b$

```
1: procedure EVKLID( $a, b$ )
2:   while  $b \neq 0$  do
3:      $t \leftarrow b$ 
4:      $b \leftarrow a \bmod b$ 
5:      $a \leftarrow t$ 
6:   end while
7:   return  $a$ 
8: end procedure
```

---



## Poglavje 7

# Matematično okolje in sklicevanje na besedilne konstrukte

Poglejmo nekaj primerov pisanja enačb, sklicevanj nanje, podajanja dokazov, matrik, razbijanje enačb in nekaj splošnih tipografskih pravil v matematiki.

**Izrek 7.1** *Za vsako naravno število  $n$  velja*

$$n < 2^n. \tag{7.1}$$

*Dokaz.* Dokazovanje z indukcijo zahteva, da neenakost (7.1) najprej preverimo za najmanjše naravno število – 0. Res, ker je  $0 < 1 = 2^0$ , je neenačba (7.1) za  $n = 0$  izpolnjena.

Enačbo (7.1) smo referencirali z `\eqref{eq:1}` in ne z `\ref{eq:1}`, saj nam slednji ne ovije reference v oklepaje. Brez oklepajev bi bralec lahko referenco na enačbo zamenjal z referenco na poglavje, z oglatimi oklepaji pa na sklic v literaturi.

Za potrebe primera nadaljujmo z indukcijskim korakom. S predpostavko, da je neenakost (7.1) veljavna pri nekem naravnem številu  $n$ , je potrebno pokazati, da je ista neenakost v veljavi tudi pri njegovem nasledniku – naravnem

številu  $n + 1$ . Računajmo.

$$n + 1 < 2^n + 1 \quad (7.2)$$

$$\leq 2^n + 2^n \quad (7.3)$$

$$= 2^{n+1}$$

Neenakost (7.2) je posledica indukcijske predpostavke, neenakost (7.3) pa enostavno dejstvo, da je za vsako naravno število  $n$  izraz  $2^n$  vsaj tako velik kot 1. S tem je dokaz Izreka 7.1 zaključen.  $\square$

Ste opazili kako so indukcijski koraki lepo poravnani? Uporabili smo pomemben `align`. Ste opazili, da smo stavek pri neenakosti 7.1 končali s piko? V primeru, da je enačba del povedi, kot npr.

$$y = 2^x, \quad (7.4)$$

pa seveda dodamo vejico v samo enačbo, kot je prikazno pri (7.4).

Za kakovostno tehnično pisanje pa moramo obvladati še zapis matrik in obdelavo zelo dolgih enačb.

## 7.1 Matrike

Za zapis matrik uporabljamo okolja iz paketa `amsmath`. Najpogosteje uporabljamo okolje `bmatrix` za matrike z oglatimi oklepaji ali `pmatrix` za matrike z okroglimi oklepaji.

Primer zapisa matrike  $A$ :

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}. \quad (7.5)$$

Pri tem elemente v vrstici ločimo z znakom `&`, vrstice pa zaključimo z `\\`. Za različne vrste pik uporabljamo ukaze `\dots` (vodoravne), `\vdots` (navpične) in `\ddots` (diagonalne).

## 7.2 Razbijanje dolgih enačb

Kadar je enačba predolga za eno vrstico, uporabimo okolje `split` znotraj okolja `equation`. To nam omogoča, da enačbo razbijemo na več vrstic, medtem ko celoten blok obdrži le eno skupno številko.

$$\begin{aligned} f(x) &= (a + b)^4 \\ &= a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4. \end{aligned} \tag{7.6}$$

Znak `&` določa točko poravnave (običajno pri enačanju), `\\` pa prehod v novo vrstico.

## 7.3 Sistemi enačb in primeri

Za zapis funkcij, ki so definirane po delih, ali za sisteme enačb z enim velikim oklepajem uporabljamo okolje `cases`:

$$|x| = \begin{cases} x, & \text{če je } x \geq 0, \\ -x, & \text{če je } x < 0. \end{cases} \tag{7.7}$$

Opazimo uporabo ukaza `\text{...}`, ki omogoča vstavljanje običajnega besedila (z vejicami in presledki) neposredno v matematično okolje, kar ohranja pravilno pokončno pisavo.

## 7.4 Splošna tipografska pravila v matematiki

Pri pisanju matematike upoštevamo naslednja pravila:

- **Vejice in pike:** Enačbe so del povedi, zato morajo vsebovati ločila. Če se poved konča z enačbo, na koncu enačbe stoji pika.
- **Standardne funkcije:** Funkcije, kot so `sin`, `cos`, `ln`, `max`, pišemo s pokončno pisavo z uporabo ustreznih ukazov (npr. `\sin`), ne pa kot *sin* (v ležeči pisavi), kar bi nakazovalo produkt spremenljivk *s*, *i* in *n*.

- **Presledki:**  $\text{\LaTeX}$  samodejno upravlja s presledki v matematiki. Ročno poseganje običajno ni potrebno, razen v redkih primerih, kjer uporabimo npr.  $\backslash$ , za majhen premik.

## Poglavje 8

### Kaj pa literatura?

Kot smo omenili že v uvodu, je pravi način za citiranje literature uporaba `BIBLATEX`a [4]. `BIBLATEX` zagotovi, da pri določeni vrsti literature ne izpustimo nobene obvezne informacije in da vse informacije dosledno navajamo na enak način in po istem vrstnem redu. `BIBLATEX` je nadgradnja starejšega sistema `BIBTEX`. Novejši sistem je bolj prilagojen slovenščini in navajanju spletnih virov. Sicer pa so starejše datoteke `.bib` kompatibilne z `BIBLATEX`om.

Osnovna ideja `BIBLATEX`a je, da vse informacije o literaturi zapisujemo v posebno datoteko, v našem primeru je to `literatura.bib`. Vsakemu viru v tej datoteki določimo simbolično ime. V našem primeru je v tej datoteki nekaj najbolj značilnih zvrsti literature, kot so knjige [8], članki v revijah [13] in zbornikih konferenc [5], poglavja v knjigah [10], spletni viri [12, 14], tehnično poročilo [1], diplome [2] itd. Diploma [2] iz leta 1990 je bila prva diploma na tedanji Fakulteti za elektrotehniko in računalništvo, ki je bila oblikovana z `LATEX`om. Reference, ki so na spletnih straneh arhivirane v elektronski obliki, imajo običajno številko DOI (<http://dx.doi.org>), ki jo zato tudi vključimo v izpis literature in tako bralcu elektronske verzije naše publikacije ponudimo neposredno povezavo do elektronske kopije te reference.

Po vsaki spremembi pri sklicu na literaturo moramo najprej prevesti izvirno besedilo s prevajalnikom `LATEX`, nato s prevajalnikom `BIBLATEX`, ki ustvari datoteko `vzorec_dip_Seminar.bbl`, in nato še dvakrat s prevajal-

nikom  $\text{\LaTeX}$ . V okolju Overleaf je to večkratno prevajanje z različnimi prevajalniki uporabniku skrito. Zato tudi začetnim uporabnikom  $\text{\LaTeX}$ a svetujemo uporabo Overleafa.

Kako se spisek literature nato izpiše (ali so posamezni viri razvrščeni po vrstnem redu sklicevanja, ali po abecedi priimkov prvih avtorjev, ali se imena avtorjev pišejo pred priimki itd.) je odvisno od parametrov paketa  $\text{BIB}\text{\LaTeX}$ . V diplomu bomo uporabili parametre `style=numeric`, kar pomeni, da bodo sklici na literaturo v besedilu označeni z zaporednimi številkami, za vrstni red izpisa referenc pa `sorting=nty`, kar pomeni, da bodo reference urejene po priimkih prvih avtorjev, nato po naslovu reference in nazadnje po letu izdaje [3]. Zato je potrebno pri določenih zvrsteh literature, ki nima avtorjev, dodati parameter `key`, ki določi vrstni red vira po abecedi.

Ko začenjamo uporabljati  $\text{BIB}\text{\LaTeX}$  je lažje, če za urejanje datoteke `.bib` uporabljamo kar isti urejevalnik kot za urejanje datotek `.tex`, čeprav obstajajo tudi posebni urejevalniki oziroma programi za delo z datotekami `.bib`.

Le če se na določen vir v besedilu tudi sklicujemo, se bo ta vir pojavil tudi v spisku literature. Tako je avtomatično zagotovljeno, da se na vsak vir v seznamu literature tudi sklicujemo v besedilu diplome. V datoteki `.bib` imamo sicer lahko veliko več virov za literaturo, kot jih bomo uporabili v diplomu.

## 8.1 Zbiranje virov za seznam literature

Vire v formatu `.bib` lahko enostavno poiščemo in prekopiramo iz spletnih strani založnikov ali različnih akademskih spletnih portalov za iskanje znanstvene literature. Izvoz referenc v Google učenjaku še dodatno poenostavimo, če v nastavitvah izberemo  $\text{BIB}\text{\LaTeX}$  kot želeni format za izvoz navedb. Navedbe, ki jih prekopiramo iz Google učenjaka in drugih podobnih akademskih portalov, moramo pred uporabo nujno preveriti, saj so taki navedki pogosto generirani povsem avtomatično in lahko vsebujejo napačne ali nepopolne podatke. Najpogosteje je napačen tip publikacije! Pri velikih jezikovnih



modelih pa je lahko celotna referenca izmišljena!

Pri sklicevanju na literaturo na koncu stavka moramo paziti, da je pika po ukazu `\cite{ }`. Da  $\text{\LaTeX}$  ne bi delil vrstice ravno tako, da bi sklic na literaturo v oglatih oklepajih začel novo vrstico, moramo pred sklicem na literaturo dodati nedeljiv presledek (tilda): `~\cite{ }` in ne navadnega presledka!

Običajno se v besedilu sklicujemo na nek vir ali več virov na koncu trdilnega stavka. Kadar pa omenimo avtorja nekega vira, pa sklic običajno vstavimo za njegovim priimkom.

Dandanes se skoraj vsi pri iskanju informacij vedno najprej lotimo iskanja preko spleta ali velikih jezikovnih modelov. Rezultati takega iskanja pa so pogosto spletne strani, ki danes obstajajo, jutri pa jih morda ne bo več, ali pa vsaj ne v taki obliki, kot smo jo prebrali. Smisel navajanja literature pa je, da tudi po dolгих letih nekdo, ki bo bral vašo diplomо, lahko poišče vire, ki jih navajate v diplomі.

Znanstveni rezultati, ki so objavljeni v obliki recenziranih člankov, bodisi v konferenčnih zbornikih, še boljе pa v znanstvenih revijah, so veliko bolj izčiščeni in zanesljiv vir informacij, saj so taki članki šli skozi recenzijske postopke. Predvsem pa so taki članki stabilen vir informacij, saj se načeloma po njihovi objavi ne spreminjajo več. Skoraj vsi ti članki so dandanes dosegljivi tudi v elektronski obliki, bodisi v arhivih založnikov, univerzitetnih repozitorijih ali tudi na osebnih spletnih straneh njihovih avtorjev. Zato na spletu začnemo iskati vire za strokovna besedila predvsem preko akademskih spletnih portalov, kot so npr. Google učenjak, Research Gate ali Academia, saj so na teh portalih rezultati iskanja le akademske publikacije. Pri velikih jezikovnih modelih bodite pozorni, saj so viri lahko izmišljeni.

Če je za dostop do nekega članka potrebno plačati, se obrnemo za pomoč in dodatne informacije na našо knjižnico.

Za označevanje člankov, ki so na voljo v elektronski obliki, se je v zadnjem času uveljavila oznaka DOI (<https://www.doi.org>), kar močno olajša iskanje teh referenc na spletu. Založniki tudi starejšim člankom, ki so na voljo v

elektronski obliki, za nazaj določajo oznake DOI. Zato poskusite poiskati ustrezno oznako DOI za vsak članek, ki ga citirate in jo vključite v seznam literature.

Če res ne gre drugače, pa je pomembno, da pri sklicevanju na običajni spletni vir vedno navedemo tudi datum, kdaj smo dostopali do tega vira.

Z uporabo BIB<sub>Λ</sub>T<sub>E</sub>Xa je možno natisniti seznam literature posebej za določene vrste referenc, na primer za članke v znanstvenih revijah, članke v konferenčnih zbornikih in poglavja v knjigah, kot je prikazano tudi v tem vzorcu diplome.

## Poglavje 9

# Sklepne ugotovitve

Uporaba L<sup>A</sup>T<sub>E</sub>Xa in BIBL<sup>A</sup>T<sub>E</sub>Xa je pri diplomski nalogi **obvezna**!

**Preden diplomo oddate mentorjem, še enkrat preverite:**

- pravilno rabo elementov (tako L<sup>A</sup>T<sub>E</sub>Xgradnikov kot tudi ločil kratic, pomišljajev itd.),
- slovnico (lahko si pomagate s katerim od orodij),
- da ni nobeno besedilo prilepljeno direktno iz velikega jezikovnega modela (čisto vse morate prebrati in preveriti, če že ne popraviti),
- da je dovolj slikovnih elementov (preveč suhoparnega besedila ni za nikogar koristno),
- da je vsaka slika in tabela ne samo podnaslovljena, ampak tudi referencirana v besedilu.

**Preden diplomo oddate na sistemu STUDIS, še enkrat preverite:**

- če je izpisana pravilna smer študija, vaše ime, ime in naziv mentorja in somentorja,
- če so vse besede pravilno deljene in da ne segajo preko desnega roba (poravnava po vrsticah lahko kontrolirate tako, da izvirno datoteko enkrat testno prevedete z opcijo **draft**, kar vam pokaže predolge vrstice),

- če ste vključili pravilne verzije slik, predvsem pri grafih in diagramih vektorske (.pdf ali .eps) in ne rasterskih (npr. .jpg ali .png),
- če ste naslovili vse napake in opozorila, ki se lahko pojavijo pri tvorjenju dokumenta (bodite pozorni predvsem na manjkajoče citate, ki jih lahko poiščete tudi z iskanjem vprašajev v končnem dokumentu).

# Literatura

- [1] Michael Riis Andersen in sod. *Kinect depth sensor evaluation for computer vision applications*. 6. Department of Engineering, Aarhus University, 2012. URL: <https://tidsskrift.dk/ece/article/view/21221> (pridobljeno 10. 5. 2021).
- [2] Andreja Balon. *Vizualizacija*. Diplomaska naloga. Fakulteta za elektrotehniko in računalništvo, Univerza v Ljubljani, 1990.
- [3] *BibLaTeX – Sophisticated Bibliographies in LaTeX*. URL: <https://ctan.org/pkg/biblatex?lang=en> (pridobljeno 11. 5. 2021).
- [4] *Bibliography management with BibLaTeX*. 1988. URL: [https://www.overleaf.com/learn/latex/Bibliography\\_management\\_with\\_biblatex](https://www.overleaf.com/learn/latex/Bibliography_management_with_biblatex) (pridobljeno 7. 5. 2021).
- [5] Peter Ciuha, Bojan Klemenc in Franc Solina. „Visualization of concurrent tones in music with colours“. V: *Proceedings of the 18th ACM international conference on Multimedia*. ACM. Firenze, 2010, str. 1677–1680. DOI: 10.1145/1873951.1874320.
- [6] Slovensko društvo Informatika. *Islovar*. URL: <http://www.islovar.org/islovar/islovar> (pridobljeno 10. 6. 2021).
- [7] Donald E Knuth in Peter B Bendix. „Simple word problems in universal algebras“. V: *Automation of Reasoning: Classical papers on computational logic 1957–1966*. Ur. Jörg H. Siekmann in Graham Wrightson. Springer, 1983, str. 342–376. DOI: 10.1007/978-3-642-81955-1\_23.

- 
- [8] Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, 1986.
- [9] Tobias Oetiker in sod. *Ne najkrajši uvod v L<sup>A</sup>T<sub>E</sub>X2<sub>ε</sub>*. 2006. URL: <http://www-lp.fmf.uni-lj.si/plestenjak/vaje/latex/lshort.pdf> (pridobljeno 1. 10. 2016).
- [10] Peter Peer in Borut Batagelj. „Art—A Perfect Testbed for Computer Vision Related Research“. V: *Recent Advances in Multimedia Signal Processing and Communications*. Springer, 2009, str. 611–629. DOI: 10.1007/978-3-642-02900-4\_23.
- [11] ZRC SAZU. *Slovarji Inštituta za slovenski jezik Frana Ramovša ZRC SAZU*. URL: <https://fran.si> (pridobljeno 14. 6. 2022).
- [12] *SLOVARJI.SI seznam slovenskih spletnih slovarjev*. URL: <https://www.slovarji.si> (pridobljeno 11. 5. 2021).
- [13] Franc Solina. „15 seconds of fame“. V: *Leonardo* 37.2 (2004), str. 105–110. DOI: 10.1162/0024094041139274.
- [14] Franc Solina. *Light Fountain II – Galaxy*. 2015. URL: <https://youtu.be/y6NAiX1Nm20> (pridobljeno 9. 6. 2021).
- [15] Franc Solina. *Skulpture/Sculptures 2012-2020, 2. izdaja / 2nd edition*. Ljubljana: Društvo likovnih umetnikov Ljubljana, Založba UL FRI, 2021. DOI: 10.51939/0001.
- [16] Franc Solina in Blaž Meden. „Light fountain – a virtually enhanced stone sculpture“. V: *Digital Creativity* 28.2 (2017), str. 89–102. DOI: 10.1080/14626268.2016.1258422.
- [17] Slovensko društvo za razpoznavanje vzorcev. *Slovar, Razpoznavanje vzorcev*. URL: <https://slovar.vicos.si> (pridobljeno 10. 6. 2021).
- [18] Wikipedia contributors. *Donald Knuth — Wikipedia, The Free Encyclopedia*. 2021. URL: [https://en.wikipedia.org/w/index.php?title=Donald\\_Knuth&oldid=1020717520](https://en.wikipedia.org/w/index.php?title=Donald_Knuth&oldid=1020717520) (pridobljeno 7. 5. 2021).